Tehnike **<u>nižeg (primitivnog) nivoa kompaktovanja</u>** koda:

- **ZAMENI** redosled zadate instrukciuje sa zadatim NOP, na postojećim adresama unutar segmenta
- **IZBACI** zadati NOP
- **ZAMENI-PRETHODNI** – zameni redosled zadate instrukcije segmenta sa zadatim NOP iz prethodnog segmenta
- **ZAMENI-NAREDNI** – zameni redosled zadate instrukcije segmenta sa zadatim NOP iz narednog segmenta
- **PREIMENUJ** resurs (akumulator) sa korišćenog imena na slobodno
- **MUTIRAJ** instrukciju

Tehnike **<u>višeg nivoa kompaktovanja</u>** koda:

- **TRAŽI-UNAPRED** – ova tehnika višeg nivoa koristi primitivne ZAMENI i IZBACI.
  Polazi od početka programskog segmenta i u smeru njegovog izvršenja (unapred) traži NOP instrukciju
- **TRAŽI-UNAZAD** - ova tehnika višeg nivoa koristi primitivne ZAMENI i IZBACI.
  Polazi od kraja programskog segmenta i u smeru početka (unazad) traži NOP instrukciju
- **KOMPAKTUJ-SA-PRETHODNIM** – ova tehnika višeg nivoa koristi primitivne ZAMENI-PRETHODNI i IZBACI
- **KOMPAKTUJ-SA-NAREDNIM** – ova tehnika višeg nivoa koristi primitivne ZAMENI-NAREDNI i IZBACI

Data su dva segmenta asemblerskog koda. Ispod svakog treba napisati simbole koje taj segment definiše i simbole koje taj segment potražuje
u **.data** se samo definiše
u **.text** može da se definiše, a i da se potražuje (jump – potražuje se labela, add – potražuju se operandi)

Sistemka programska podrška je sistemski softver. Ona podrazumeva pomoćne programe tj. podsisteme (asembler, kompajler, punjač ...)

Navesti 4 lingvistička nivoa programske podrške

1. Upravljanje resurima
2. Direktna komunikacija sa računarskim sistemom
3. Indirektna komunikacija sa računarskim sistemom
4. Skup aplikacionih programa

Navesti 3 tehnike za definisanje novih lingvističkih nivoa programske podrške
1. **proširenje** – uvođenje novog lingvističkog nivoa (npr. biblioteke)
2. **prevođenje** – kompajliranje – prevođenje novog jezika na jezik osnovnog sistema (prevodi se sa jednog lingvističkog nivoa na drugi)
3. **interpretacija** – podrazumeva da su faze prevođenja i izvršenja vremenski zavisne (funkcioniše kao prevođenje samo uvodi i izvršavanje)

Regularan izraz za realne brojeve
(“-“ ? [0-9]* “.“ [0-9]+)| (“-“ ? [0-9]+ “.“ [0-9]*)

* - 0 ili više puta
\+ - 1 ili više puta

Četiri uobičajena koraka u pisanju paralelnih programa
1. **dekompozicija**
2. **dodela** (zadataka procesima)
3. **orkestracija** (komunikacija između procesa)
4. **preslikavanje** (mapiranje)

Tri oblika dekompozicije u toku...
1. Zadataka
2. Podataka
3. Protočne obrade

Tri osnovna zadatka leksičke analize:
1. podela polaznog programa na niz osnovnih simbola - tokena
2. formiranje tabele literala i identifikatora
3. formiranje tabele uniformnih simbola

Zadatak sintaksne analize
- Prepoznaje rečenice i interpretira njihovo značenje
- Ukazuje na greške i omogućava kompajleru nastavak traženja drugih grešaka

---

Tehnike poboljšanja generisanog koda:
1. Traženje mogućnosti primene složenijih oblika MAC mašinske instrukcije
2. Traženje mogućnosti za korišćenje adresnih registara
3. Izbacivanje suvišnih mašinskih instrukcija

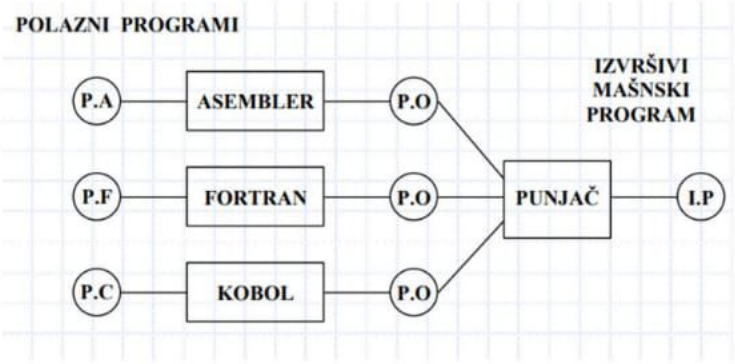
Šta je programski trag?
- Trag predstavlja niz iskaza koji se mogu uzastopno izvršavati tokom izvršavanja programa. Može sadržati i uslovne skokove (vrv kad uslov nije ispunjen, pa neće skočiti)

---

p = q	p = &q	*p = *q
(p, 0) – (q, 0)	(p, 0) – (q, -1)	(p, 1) – (q, 1)

PUNJAČI

PUNjAČ (*loader*): Program koji smešta druge programe u memoriju



Funkcije punjača

1. Alokacija - Dodela memorijskog prostora programima
2. Povezivanje - Određivanje vrednosti simboličkih referenci između relokatibilnih programa
3. Relokacija - Podešavanje svih adresno osetljivih lokacija na odgovarajući dodeljeni prostor
4. Punjenje - Fizički prenos mašinskih instrukcija i podataka u memoriju
5. Povezivanje programa sa sistemskim bibliotekama

Tipovi punjača

- Prevedi i kreni.
- Apsolutni.
- BSS.
- Relokatibilni sa direktnim povezivanjem.
- Povezivač i punjač (apsolutne memorijske slike i editor povezivanja).
- Punjač prekrivača.
- Dinamički povezivač.

2 faze punjenja:
Glavna uloga punjača u PRVOJ FAZI je dodela i pridruživanje memorije svakom segmentu programa i biblioteke i formiranje tabele simbola u koju se unose globalni simboli i apsolutne adrese.
Glavna funkcija DRUGE FAZE je punjenje stvarnog programa i obavljanje relokacione modifikacije neke adresne komponente koja traži tu modifikaciju

Realizuje se u 2 prolaza.
U 1. prolazu se analiziraju relokatibilne objektne datoteke i prave se kopije .data i .text sekcija u lokalnoj memoriji (1. alokacija) i popunjava se lista globalnih simbola, zajednička za sve ulazne datoteke (2. povezivanje).
U 2. prolazu se redom kopije sekcija spajaju u zajedničku celinu (3. relokacija, 4. punjenje, 5. povezivanje)

Apsolutni punjač

- Asembler formira u celini prevedeni program.
- To znači da su adrese prvih instrukcija u programu i proceduri definisane u vreme asembliranja.
- Apsolutni punjač jednostavno unosi sa spoljne memorije program u definisane memorijske lokacije.

Nedostaci:

- Na određeni način negira suštinu asemblera.
- Nemoguće je rešiti problem biblioteka ili programa koji se sastoji od više nezavisnih modula

Prednosti:

- asembler formira u celini prevedeni program, što znači da su adrese prvih instrukija u programu i proceduri definisane u vreme asembliranja

BSS punjač

Nedostaci:

- Vektor prelaza zauzima prostor u memoriji, a koristi se samo za povezivanje.
- Obrađuju segmente procedure, ali ne olakšavaju pristup segmentima deljivih podataka.

Prednosti:

- Omogućava punjenje više segmenata programa nad samo jednim zajedničkim segmentom podataka.
- Izlaz iz asemblera je prevedeni program i informacija o svim drugim programima na koje se on poziva.
- Navodi se i informacija o tome koje reference treba menjati (relokaciona informacija).

E&L - Executable and linkable format opisuje 3 tipa obj. datoteka

1. relokatibilna datoteka
2. izvršna datoteka
3. deljena datoteka

Kako su definisane tačka izvršavanja i tačka pristupa u fazi pripreme koda za protočnu obradu?

- Tačka izvršavanja ep je jednaka indeksu instrukcije u nizu instrukcija, a tačkapristupa operadni se dobija tako što se na ep doda redni broj faze protočne obrade u kojoj se pristupa operandi.

Dve grupe pristupa kod aspekta optimizacije:

1. slagajući raspoređivač
2. perkolokacioni raspoređivač

Osnovni algoritmi za dodelu resursa

- formiraj
- uprosti
- prelij
- izaberi

ponovi

ŠKOLSKI KOMPAJLER

Školski kompajler - prevodi programe napisane u pravolinijskom programskom jeziku na asemblerski jezik LPRS procesora
-ne podržava množenje i deljenje
-print naredba se svodi na upisivanje vrednosti na adresu 31
-u 23 datoteke ispod 1500 linija koda ??????????????
Ograničenja:

1. nema grananja
2. slaba detekcija grešaka u ulaznoj datoteci i oporavak od njih
3. nema oslobađanja dinamički zauzetih objekata
4. postoji samo jedan tip u ulaznom jeziku – celobrojni
5. nema provere semantike
6. generiše se asembler bez segmenata
7. postoji samo 1 doseg (i nema funkcija)
8. svi neposredni operandi mogu zauzimati najviše 6 bita ....

Faze u školskom kompajleru:

1. Leksička analiza
2. Sintaksna analiza
3. Generisanje instrukcija
4. Pridruživanje memorije
5. Asembliranje

Leksička analiza – 3 osnovna zadatka:

1. podela polaznog programa na niz osnovnih simbola – tokena
2. formiranje tabele literala i identifikatora
3. formiranje tabele uniformnih simbola

Tri mašinski nezavisne optimizacije kod školskog kompajlera su:

1. eliminacija zajedničkih podizraza
2. prethodno izračunavanje vrednosti izraza
3. uočavanje invarijantnog računanja unutar ciklusa

Sintaksna analiza prepoznaje rečenice (sintaksne konstrukcije) i interpretira značenje tih konstrukcija. Ona ukazuje i na uočene greške i omogućava kompajleru da nastavi pretraživanje drugih grešaka.

Pridruživanje – dodela memorije – 4 zadatka:

1. dodela memorije svim promenljivim
2. dodela memorije svim privremenim lokacijlama koje su potrebne za međurezultate
3. dodela memorije literalima
4. obezbeđivanje inicijalizacije odgovarajućih lokacija

Generisanje instrukcija obavlja prevođenje operacija prelaznog oblika u operacije odredišnog jezika
Asembliranje – da bi se dobio mašinski kod, treba izlaz iz školskog kompajlera proslediti asemblerskom prevodiocu za LPRS procesor

OPTIMIZUJUĆI KOMPAJLER

Zarad povećanja brzine pristupa, tabela simbola kod optimizujućeg kompajlera se realizuje kao Hash tabela.

Faze u zadnjem delu visoko optimizujućeg kompajlera?

- Izbor instrukcija
- Mašinski zavisna optimizacija
- Analiza toka upravljanja
- Analiza toka podataka
- Dodela resursa
- Prilagođavanje koda protočnoj strukturi
- Generisanje koda
- Povezivanje

Šta je ulaz u fazu izbora instrukcija kod optimizujućeg kompajlera?

- Ulaz u fazu izbora instrukcija je stablo međukoda

Šta je izlaz iz faze analize toka upravljanja, a šta iz faze analize toka podataka kod optimizujućeg kompajlera?
Izlaz iz faze analize toka upravljanja je graf toka, a iz faze analize toka podataka je graf smetnji

Šta je izlaz iz faze sintaksne analize kod optimizujućeg kompajlera?
Izlaz iz sintaksne analize optimizujućeg kompajlera je apstraktna analiza.

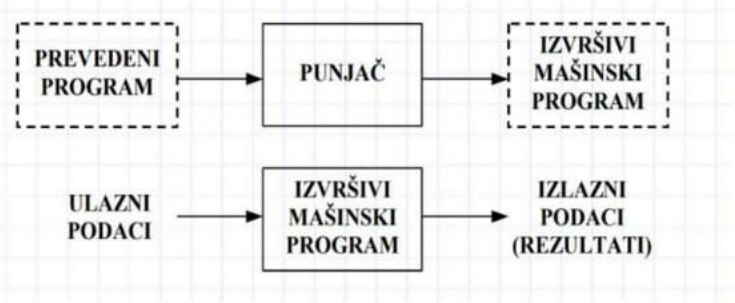
Navesti dve glavne grupe klasa apstraktne sintakse

1. iskazi
2. izrazi

Šta su osnovni zadaci statičke provere semantike?
Statička provera semantike podrazumeva proveru zadovoljenosti pravila kao što su: provera tipova svih operanada u izrazima (svi operandi moraju biti istog tipa), provera broja parametara kod poziva funkcije (broj parametara pri pozivu funkcije mora biti jednak broju parametara u definiciji funkcije).

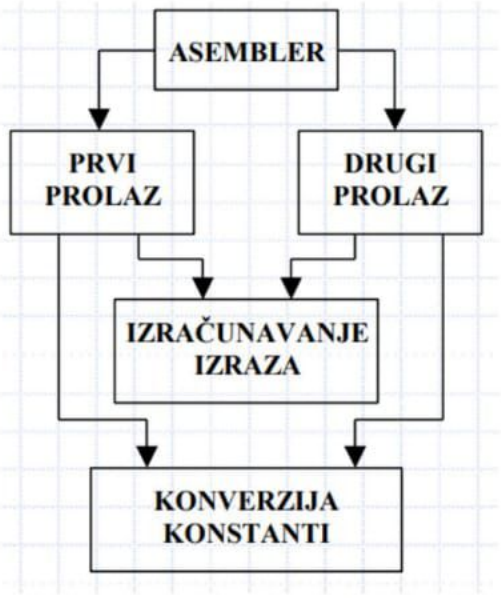
ASEMBLER

Asembler – program koji prevodi izvorni program napisan na asemblerskom jeziku u program na mašinskom jeziku



Četiri osnovna modula asemblera:

- Modul za prvi prolaz,
- Modul za drugi prolaz,
- Modul za izračunavanje izraza,
  - U I prolazu za obradu EQU,
  - u II prolazu za određivanje vrednosti operanada
- Modul za konverziju konstanti.
  - U I prolazu za obradu literala,
  - u II prolazu za obradu α: DATA β i literala



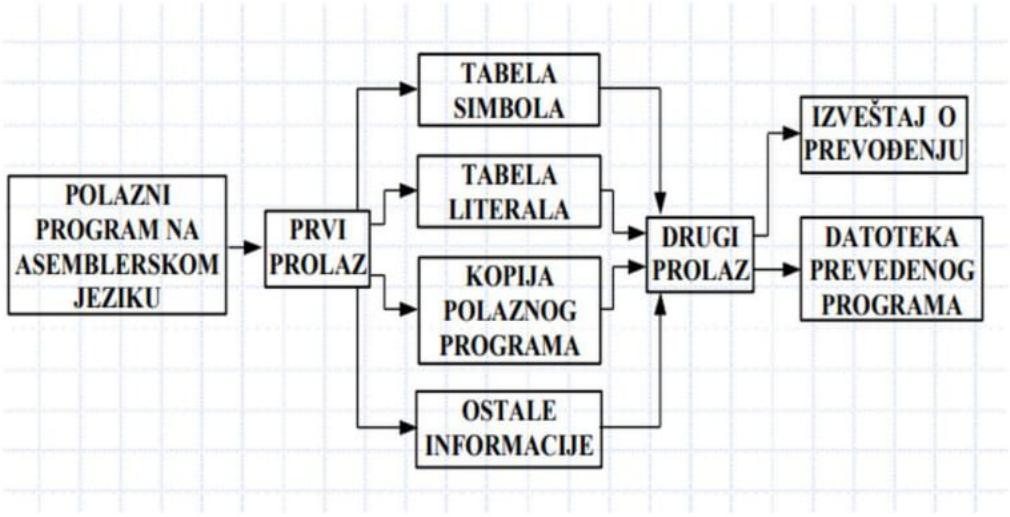


Tabela simbola (simboli – labele i promenljive)

- Definicija simbola predstavlja proces pridruživanja simbola broju koji predstavlja njegovu vrednost.
- Prvi prolaz dodeljuje memorijsku lokaciju svakoj instrukciji.
- TABELA SIMBOLA služi za upisivanje simbola i njima pridruženih vrednosti.

Tabela literala

- Literali: specijalna vrsta simbola koji se odnose na konstante.
- Prvi prolaz dodeljuje memorijske lokacije i svim literalima u programu.
- TABELA LITERALA sadrži adresu i binarni zapis konstante koja odgovara datom literalu.

Modul za prvi prolaz asemblera

- Treba da definiše sve simbole Zato dodeljuje memorijske lokacije za:
  - Za svaku mašinsku instrukciju
  - Za svaku konstantu (koje generiše pseudo operacija α: DATA β)
  - Za svaki memorijski blok (rezervisan iskazom α: DATA n)
  - Za svaku konstantu (koju generiše literal)

Modul za drugi prolaz asemblera
Izlazi:

- datoteku prevedenog programa (mašinski kod .obj)
- izveštaj o prevođenju (.lst)

U 2. prolazu asemblera obavlja se: zamena simboličkih imena labela njihovim pravim vrednostima

<u>Projektovanje makroasemblera</u>

Dva potrebna proširenja asemblera da bi se napravio makro asembler

1. da prepozna makroinsrtukcija
2. da proširi makroinstrukciju

Da bi to uradio, on najpre mora da pronađe makrodefinicjiju koja odgovara makroinstrukciji.
Makrodefinicija počinje sa MACRO, a završava se ENDM pseudo instrukcijom

Kako se vrši prepoznavanje makroinstrukcije?

- Najprostiji način je dodati vrstu u tabeli koda operacije. Ova vrsta sadrži naziv makroinstrukcije i ukazivač na odgovarajuće makrodefinicije. Definicije makroa se čuvaju u tabeli makrodefinicija

Primeri zadataka – asembler – popunjavanje tabele simbola

<table>
<tr><th></th><th>zadatak</th><th>Rešenje</th></tr>
<tr><td>1.</td><td>

```
niz:    .word 9 123 234 53 1
Begin:
        subi    $t2, $t2, 5 * 7
.space 24
        addi    $t1, $0, niz + 3
        b Begin - 3
```

</td><td>Koliko ima simbola?
2 (niz, Begin)
Koliko ima literala?
5 (9, 123, 234, 53, 1)
Koliko puta će se pozvati model za izračunavanje izraza u 2. prolazu asemblera?</td></tr>
<tr><td>2.</td><td colspan="2">Na osnovu asemblerskog koda, popuni tabelu simbola. Širina memorije za dati primer je 8 bita, veličina podataka 24 bita (.word = 24 bita), a veličina svake instrukcije 16 bita.


```
.data
tmp:    .word 100 0
a:      .word 70 3
val:    .word 4 6

.text
main:
        la      $t0, tmp
        lw      $t1, 0($t0)
        li      $t0, 5
        li      $t2, -5
lab:
        add     $t1, $t1, $t0
        addi    $t2, $t2,  1
        blez    $t2, lab
        nop
        la      $t0, a
        sw      $t1, 0($t0)
exit:
        nop
```

<table>
<tr><th>simbol</th><th>vrednost</th><th>sekcija</th></tr>
<tr><td>tmp</td><td></td><td></td></tr>
<tr><td>a</td><td></td><td></td></tr>
<tr><td>val</td><td></td><td></td></tr>
<tr><td>main</td><td></td><td></td></tr>
<tr><td>lab</td><td></td><td></td></tr>
<tr><td>exit</td><td></td><td></td></tr>
</table>

</td></tr>
</table>

3. Na osnovu asemblerskog koda, popuni tabelu simbola. Širina memorije za dati primer je 8 bita, veličina podataka 24 bita (.word = 24 bita), a veličina svake instrukcije 16 bita.

```
.data
tmp:    .word 100
a:      .word 70
val:    .word 4

.text
main:
        la      $t0, tmp
        lw      $t1, 0($t0)
        li      $t0, 5
        li      $t2, -5
lab:
        add     $t1, $t1, $t0
        addi    $t2, $t2,  1
        blez    $t2, lab
        nop
        la      $t0, a
        sw      $t1, 0($t0)
exit:
        nop
```

simbol	vrednost	sekcija
tmp		
a		
val		
main		
lab		
exit		

PETLJE

1. veoma dobro definisana	poznat je naziv i početna vrednost indeksne promenljive poznat je korak iteracije poznat je broj iteracija pre prevođenja izvornog koda	for (int i = 6; i < 100; i += 5) i=0; while (i < 10) { i++; }
2. dobro definisana	poznat je naziv i početna vrednost indeksne promenljive poznat je korak iteracije poznat je broj iteracija, ali tek prilikom pokretanja petlje	n = 10; for (int i = 6; i < n; i += 5)
3. loše definisana	granica se menja pri svakoj iteraciji	for (i = 6; i < foo(y); i ++) { y+=i; }

MEĐUKOD

Dobar međukod zadovoljava sledeće osobine:

- pogodan je za prevođenje u mašinski jezik bilo kog odredišnog procesora i tj. nezavisan je od izvornog koda (programskog jezika) i ciljnog procesora
- pogodan je za davanje značenja čvorovima stabla sintaksne analize (prevođenje iz stabla sintakse analize u međukod),
- čvorovi stabla međukoda treba da imaju jasna i jednostavna značenja,
    - da bi se optimizacione transformacije međukoda mogle lako specificirati i realizovati.

ADRESNO OSETLJIVE LOKACIJE

Adresno osetljive lokacije – svako b, blez, jump koje ima labelu

Primeri zadataka – adresno osetljive lokacije

<table>
<tr><th></th><th>zadatak</th><th>Rešenje</th></tr>
<tr><td>1.</td><td colspan="2">Zaokruži instrukcije koje predstavljaju adresno osetljive lokacije

main:
la $t0, 7
lab1:
lw $t1, 0($t0)
li $t0, 5
b lab2 + 1
li $t2, -5
blez lab1
lab2:
nop</td></tr>
<tr><td>2.</td><td colspan="2">Zaokruži instrukcije koje predstavljaju adresno osetljive lokacije

main:
la $t0, 7
lw $t1, 0($t0)
blez $t1, lab2
li $t0, 5
lab2:
b lab1 + 6
lab1:
li $t2, -5</td></tr>
<tr><td>3.</td><td colspan="2">Zaokruži instrukcije koje predstavljaju adresno osetljive lokacije</td></tr>
<tr><td></td><td>main:
la $t0, 7
lw $t1, 0($t0)
blez $t1, l1
li $t0, 5
l2:
b foo + 6
l1:
li $t2, -5</td><td>foo:
addi $t1, $t1
mult $t1, $t1, $t0
blez $t1, l3
b main
l3:
li $t2, -4</td></tr>
</table>

OSNOVNI BLOK

Osnovni programski blok se definiše kao niz susednih iskaza koji:

- Ne sadrže labelu, osim možda na samom početku
- Ne sadrže skokove, osim možda na samom kraju

Osnovni blok je niz susednih iskaza koji se uvek izvršavaju u paketu, bez skokova i mogućih uskoka.

Primeri zadataka – osnovni blok

<table>
<tr><th></th><th>zadatak</th><th>Rešenje</th></tr>
<tr><td>1.</td><td colspan="2">Sve instrukcije osnovnog bloka, sem prve i poslednje predstavljaju njegovu unutrašnjost. Koje od navedenih instrukcija međureprezentacije mogu biti u unutrašnjosti osnovnog bloka
• Pristup memoriji
• Labela
• Poziv potprograma
• Uslovni skok
• Bezuslovni skok
• Move instrukcija</td></tr>
<tr><td>2.</td><td>Dat je kod, nacrtati graf toka upravljanja i označiti osnovne blokove

int foo(int x, int y)
{
int b, z, a = 5;
if (x < 5)
z = y – a;
else
z = x;
b = 7;
return z – b;
}</td><td></td></tr>
<tr><td>3.</td><td colspan="2">Podeli na osnovne blokove

main:
la $t0, vor1
lab1:
lw $t1, 0($t0)
li $t0, 5
blez $t2, lab1
li $t2, -5
blez $t2, lab1
lab2:
nop</td></tr>
</table>

Analiza životnog veka čvora

Analiza životnog veka je neophodna da bi promenljive bez ometanja delile isti resurs
Životni vek započinje definicijom a završava se njenom poslednjom upotrebom.
Alternativna definicija: promenljiva ej živa na strelici grafa ako postoji usmerena putanja od te strelice do čvora koji koristi tu promenljivu, a da putanja ne prelazi preko nijednog čvora koji je definiše (usmerena putanja od te strelice do „use" čvora, a da ne prelazi preko nijednog „def" čvora)

Jednačine životnog veka za čvor n

$in[n] = use[n] \cup (out[n] - def[n])$ $out[n] = \cup_{succ[n]} in[s]$	in – skup promenljivih živih na ulazu out - skup promenljivih živih na izlazu def - skup promenljivih koje čvor definiše use - skup promenljivih koje čvor koristi succ – skup čvorova naslednika

Graf smetnji – nemogućnost deljenja istog registra (resursa), najčešće zbog preklapanja životnog veka promenljivih
Dva pravila za dodavanje nove smetnje u grafu smetnji:
1. za svaku definiciju promenljive A, koja nije u čvoru MOVE, dodaj smetnje između nje i svake promenljive žive na izlazu čvora
2. za MOVE instrukciju A = C dodaj smetnje između promenljive A i svake promenljive žive na izlazu čvora koji nije C

Primeri zadataka – analiza životnog veka čvora

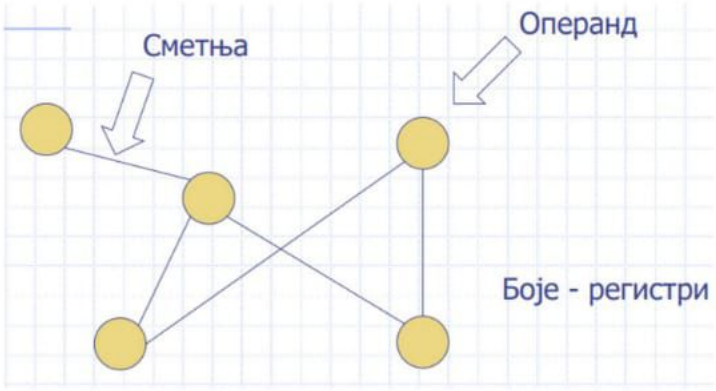
	zadatak	Rešenje

1.	Od koje do koje linije koda traje životni vek promenljive a?
	int foo(int x, int y)
	{
	int b, z, a = x – 2;
	b = 4 + a;
	b += a + y;
	z = 1 + b;
	return z – a;
	}
		start
		
		end

	Od koje do koje linije koda traje životni vek promenljive b?
	int foo(int x, int y)
	{
	int b, z, a = 5;
	if (x < 5)
	z = y – a;
	else
	z = x;
	b = 7;
	return z – b;
	}
		
		start
		end

Bojenje grafa

Bojenje grafa (čvorova grafa) je pridruživanje svakom čvoru neke boje, ali tako da susedni čvorovi ne mogu imati istu boju.
Hromatski broj grafa je najmanji broj boja koji je dovoljan da se graf oboji



k – zadati broj registara

Dva slučaja:

- Hromatski broj grafa je manji ili jednak K (broj boja <= broj raspoloživih registara)
  - Naš algoritam može da oboji dati graf
- Hromatski broj grafa je veći od K (broj boja > broj raspoloživih registara)
  - Naš algoritam ne može da oboji dati graf
  - dolazi do PRELIVANJA (engl. spill) neki operandi se moraju smestiti u neku drugu klasu resursa (npr. memorijske lokacije), što se naziva prelivanjem.

Značaj čvor >= k (broj raspoloživih boja)

Def: Rang čvora koji je veći ili jednak K (br. raspoloživih boja) je značajan rang.
Def: Za čvor čiji rang je značajan kaže se da je značajan čvor.

- Strategija Briggs – Čvorovi A i B se mogu spojiti ukoliko rezultantni čvor AB ima manje od K značajnih suseda.
- Strategija George – Čvorovi A i B se mogu spojiti ukoliko svaki sused čvora A ima smetnju sa B, ili nema značajan rang (tj. rang mu je manji od K).

Dodela resursa – bojenje grafova
Osnovni primitivni algoritmi za dodelu resursa:
1. formiraj 2. uprosti 3. prelij 4. izaberi 5. ponovi

Primeri zadataka – analiza životnog veka čvora, graf toka upravljanja, bojenje grafa smetnji

<table>
<tr><th></th><th>zadatak</th><th>Rešenje</th></tr>
<tr><td>1.</td><td>

```
int foo(int x, int y)
{
        int b, z, a = 5;
        if (x > 5)
                z = y – a;
        else
                z = x;
        b = z;
        return z – b;
}
```

	use	def	out	in

</td><td></td></tr>
<tr><td>2.</td><td>

```
int foo(int x, int y)
{
        int b, z, a = x – 2;
        b = 4 + a;
        b += a + y;
        z = 1 + b;
        return z – a;
}
```

	use	def	out	in

</td><td></td></tr>
</table>

Formalna gramatika

Formalna gramatika - formalan opis sintakse nekog jezika

ključna je činjenica da se mogu iskoristiti za saopštavanje znanja o jeziku u obliku koji je pogodan za računar

4 elementa i 1 ograničenje formalne gramatike:

1. N skup neterminalnih simbola
2. T skup terminalnih simbola
3. Σ početni simbol
4. P skup smena $\alpha \rightarrow \beta \quad \alpha, \beta \in (N \cup T), \quad \alpha \neq 0$

ograničenje: N i T su disjunktni, tj. $N \cap T = \emptyset$

Dva aspekta jezika:

1. sintaksa – od nje krećemo, ali je semantika uvek prisutna
2. semantika – značenje

(rečenica može biti sintaksno ispravna, ali neispravna semantički – npr Slon je poleteo.)

Specifikacija jezika:

- Generativna – saopštava postupak koji pravi valjane rečenice (najčešći slučaj formalne gramatike)
- Analitička – saopštava postupak koji za svaku rečenicu može da utvrdi da li je valjana

Gramatika – skup produkcija koje opisuju jezik sa jednim ili više simbola sa desne strane
Rečenična forma je bilo koja reč koja može da se razvije od polaznog sistema
Rečenica je rečenična forma koja sadrži samo terminalne simbole
Azbuka programskog jezika je konačan skup terminalnih simbola
Rečnik programskog jezika je skup svih konačnih reči azbuke

Programski jezik (α) je podskup rečnika, definisan pomoću jednog ili više znakova smene, gde smena zadatu reč zamenjuje jednom od reči u koji se on transformiše
Formalni sistem je neinterpretativan matematički sistem koji čine azbuka, aksiome i skup pravila zaključivanja.
Skup posledica formalnog sistema: skup reči koji se dobija primenom pravila zaključivanja nad skupom reči jezika.
Teza formalnog sistema: skup posledica izvedenih iz sopstvenih aksioma

Formalni sistem je odlučiv ako postoji algoritam koji omogućava određivanje da li je neka reč jezika element teze.

Gramatika(1,2,3, ... n) => jezik

Bakus-Naurova forma (BNF, nekad i Bakusova normalna forma)

BNF - notacija za izražavanje produkcije (generativnih pravila).

Svaka produkcija definiše sintaksnu klasu (neterminalni simbol).

Elementi ovog meta-jezika su:

::= predstavlja strelicu smene

< > ograđuje ime sintaksne klase

| simbol „ILI“, omogućava dodelu više smena

Primeri zadataka – formalna gramatika

	zadatak	Rešenje
1.	Data je gramatika jezika J. Ispod svake rečenice napisati da li je ta rečenica deo jezika J i ako jeste, napisati redosled primene produkcija koji generiše tu rečenicu. Početni simbol je S. 1. S → y += P 2. S → F += x – P 3. P → x – 2 4. P → F – 2 5. F → z 6. F → x – 2 Rečenice: y += x – 2 – 2 z += x – x – x – 2 – 2 z += x – p	Neterminalni: S, P, F Terminalni: y, +=, –, x, 2, z Početni simbol: S Generativna gramatika y += x – 2 – 2 jeste deo gramatike 1. – 4. – 6. z += x – x – x – 2 – 2 jeste deo gramatike 2. – 5. – 4. – 6. z += x – p nije deo gramatike
2.	Data je gramatika S → x = P S → F = 2 – P / t P → 2 – F P → F / t F → x F → 2 – P Koliko ima produkcija? Prebaci datu gramatiku u BNF notaciju?	6 produkcija BNF notacija: <S> ::= x = P| F = 2 – P / t <P> ::= 2 – F| F / t <F> ::= x | 2 – P
3.	Napisati gramatiku jezika na osnovu rečenica 1. x = y 2. tmp = 19 – 4 + abc – f 3. a = 10	S → id := E E → F – E E → F + E E → F F → id F → num

<table>
<tr><td>4.</td><td>Data je sledeća gramatika:
S → get E from S
S → begin S L
S → print E END
L → quit
L → ; S L
E → num EQ num
Odredi terminalne i neterminalne simbole</td><td>Neterminalni: S, L, E

Terminalni: get, from, begin, print, END, quit, ; , num, EQ</td></tr>
<tr><td>5.</td><td>Date su rečenice
x = 10
y = x
x = 10 + 5 + y
Dato je da postoje terminalni simboli: id, num, +, -, *, /, ; . Napisati gramatiku koja generiše date rečenice.</td><td>S → x = P;
S → y = P;
P → num
P → x
P → F
F → num + num + y</td></tr>
<tr><td>6.</td><td colspan="2">Koje od datih opcija mogu su ispravne formalne gramatike?
<table><tr><td>Terminalni:
a,b,c,+,/,d,z

Neterminalni:
x, y, a, b

Početni: y</td><td>Terminalni:
a,b,c,+,/,d

Neterminalni:
x, y, z

Početni: z</td><td>Terminalni:
a,b,c,+,/,d

Neterminalni:
x, y, z

Početni: a</td><td>Terminalni:
a,b,c,+,*

Neterminalni:
*, /

Početni: a</td></tr></table></td></tr>
</table>

8.	Transformisati datu gramatiku tako da nema konflikata $E \rightarrow a\ b\ T$ $E \rightarrow a\ c\ G$	
9.	Transformisati datu gramatiku tako da nema konflikata $E \rightarrow T$ $E \rightarrow E - T$ $E \rightarrow E + T$ $T \rightarrow num$ $T \rightarrow id$ $T \rightarrow (E)$	

Asimptotske granice - Master teorema

Θ-notacija - ASIMPTOTSKI USKO OGRANIČENJE
Θ-notacija se zasniva na odbacivanju članova nižeg reda i zanemarivanju koeficijenta uz vodeći član
Za bilo koji polinom p(n) reda d, važi da je
$p(n)= \Theta(n^d)$
	Primer: asimptotski uska granica $\Theta(n^2)$
	
	void foo() {
	m = 0;
	for (int i= 0; i<n; i++)
	for (int j= 0; j<n; j++) { m++; }
	}

O-notacija - ASIMPTOTSKI USKA GORNJA GRANICA	Primer: asimptotski uska gornja granica $O(n^2)$
	
	void foo() {
	m = 0;
	for (int i= 0; i<n; i++)
	for (int j= 0; j<n && condition; j++) { m++; }
	}

Ω-notacija - ASIMPTOTSKI USKA DONJA GRANICA	Primer: asimptotski uska donja granica $\Omega(n^2)$
	
	void foo() {
	m = 0;
	for (int i= 0; i<n; i++)
	for (int k= 0; k<n; k++)
	for (int j= 0; j<n && condition; j++) {
	m++; }
	}

o-notacija - Granica koju daje O-notacija ne mora biti asimptotski uska	Primer: asimptotska gornja granica $o(n^2)$
	
	void foo() {
	m = 0;
	for (int i= 0; i<n; i++)
	{ m++; }
	}

ω-notacija - donja granica koja nije asimptotski uska	Primer: asimptotska donja granica $\omega(n^2)$
	
	void foo() {
	m = 0;
	for (int i= 0; i<n; i++)
	for (int k= 0; k<n; k++)
	for (int f= 0; f<n; f++)
	for (int j= 0; j<n &&
	condition; j++) { m++; }
	}

Asimptotska Θ-notacija Θ(g(n)) je skup funkcija {*f(n)*} $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n),\ \forall n > n_0$ $f(n) \in \theta(g(n)) \leftrightarrow f(n) = \theta(g(n))$ *g(n)* je asimptotski usko ograničenje	$\frac{n^2}{2} - 3n = \theta(n^2) \ \rightarrow f(n) = \frac{n^2}{2} - 3n, \qquad g(n) = n^2$ $c_1 n^2 \leq \frac{n^2}{2} - 3n \leq c_2 n^2$ $c_2 \leq \frac{1}{2}, \quad c_1 \leq \frac{1}{14}$ $6n^3 \neq \theta(n^2)$
Asimptotska O-notacija O(g(n)) je skup funkcija {*f(n)*} $0 \leq f(n) \leq c\ g(n),\ \forall n > n_0$ *g(n)* je asimptotski gornje ograničenje Po teoriji skupova: $\theta(g(n)) \subseteq O(g(n))$	
Asimptotska Ω-notacija Ω (g(n)) je skup funkcija {*f(n)*} $0 \leq c\ g(n) \leq f(n),\ \forall n > n_0$ *g(n)* je asimptotski donje ograničenje Po teoriji skupova: $\theta(g(n)) \subseteq \Omega(g(n))$	
Asimptotska o-notacija o(g(n)) je skup funkcija {*f(n)*} $0 \leq f(n) \leq cg(n),\ \forall n > n_0$ razlika između O i o notacije je što o definiše gornju granicu koja nije uska, a O notacija daje granicu koja može i ne mora biti uska	$2n = o(n^2)$ $2n^2 \neq o(n^2)$ $f(n) = o(g(n)) \rightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \ \neq const.$
Asimptotska ω-notacija ω (g(n)) je skup funkcija {*f(n)*} $0 \leq c\ g(n) \leq f(n),\ \forall n > n_0$ *g(n)* je asimptotski donje ograničenje razlika između Ω i ω notacije je što ω definiše donju granicu koja nije uska, a Ω notacija daje granicu koja može i ne mora biti uska	$\frac{n^2}{2} = \omega(n)$ $\frac{n^2}{2} \neq \omega(n^2)$ $f(n) = \omega(g(n)) \rightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$

Master metoda

Recept za rešavanje rekurentne jednačine oblika

T(n) = aT(n/b) + f(n)

T(1) = d, a>=1, b>1, f(n) neka asimptotski pozitivna funkcija

Opisuje vreme izvršavanja algoritma koji deli problem veličine n na problem veličine n/b

T(n/b) - vreme izvršavanja potproblema

f(n) – pokriva cenu deljenja problema i kombinovanja rešenja

	Asimptotske granice T(n)		$n^{\log_b a}$ [ ] $f(n)$
1.	$f(n) = o\left(n^{\log_b a - \varepsilon}\right)$ $\varepsilon > 0, a \geq 1, b > 1$	$T(n) = \theta(n^{\log_b a})$	>
2.	$f(n) = \theta\left(n^{\log_b a}\right)$	$T(n) = \theta(n^{\log_b a} \cdot \log_b n)$	=
3.	$f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right)$ && $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n), \quad c < 1$	$T(n) = \theta(f(n))$	<

Primeri zadataka – Master metoda

	zadatak	Rešenje
1.	$T(n) = 9T\left(\frac{n}{3}\right) + n$	$a = 9, \quad b = 3, \quad f(n) = n$ $\log_b a = \log_3 9 = 2 \rightarrow n^{\log_b a} = n^2$ $f(n) = n = o(n^{2-\varepsilon}), \quad \varepsilon = 1 > 0$ $\rightarrow T(n) = \theta(n^{\log_b a}) = \theta(n^2)$
2.	$T(n) = T\left(\frac{2n}{3}\right) + 1$	$a = 1, \quad b = \frac{3}{2}, \quad f(n) = 1 = n^0$ $\log_b a = \log_{3/2} 1 = 0 \rightarrow n^{\log_b a} = n^0 = 1$ $f(n) = 1 = n^0 = \theta(n^0)$ $\rightarrow T(n) = \theta\left(n^{\log_b a} \cdot \log_b n\right) = \theta(\log_{3/2} n)$

3.	$T(n) = 3T\left(\frac{n}{4}\right) + n\,lgn$	$a = 3, \quad b = 4, \quad f(n) = n\lg n$ $\log_b a = \log_4 3 \approx 0.8 \rightarrow n^{\log_b a} = n^{0.8}$ $f(n) = n\lg n = \Omega(n^{0.8+\varepsilon}), \quad \varepsilon = 0.2$ zahteva proveru regularnosti Uslov regularnosti: $a \cdot f\left(\frac{n}{b}\right) \le c \cdot f(n), \quad c < 1$ $3 \cdot \frac{n}{4}\lg\frac{n}{4} \le c \cdot n\lg n$ $\frac{3}{4}\lg\frac{n}{4} \le c \cdot \lg n, \quad za\ c = \frac{3}{4} < 1$ $\rightarrow T(n) = \theta(f(n)) = \theta(n\lg n)$
4.	$T(n) = 8T\left(\frac{n}{2}\right) + 1000n^2$	$a = 8, \quad b = 2, \quad f(n) = 1000n^2$ $\log_b a = \log_2 8 = 3 \rightarrow n^{\log_b a} = n^3$ $f(n) = 1000n^2 = o(n^{3-\varepsilon}), \quad \varepsilon = 1 > 0$ $\rightarrow T(n) = \theta\left(n^{\log_b a}\right) = \theta(n^3)$
5.	$T(n) = 2T\left(\frac{n}{2}\right) + 10n$	$a = 2, \quad b = 2, \quad f(n) = 10n$ $\log_b a = \log_2 2 = 1 \rightarrow n^{\log_b a} = n$ $f(n) = 10n = \theta(n^1)$ $\rightarrow T(n) = \theta\left(n^{\log_b a} \cdot \log_b n\right) = \theta(n\log_2 n)$
6.	$T(n) = 2T\left(\frac{n}{2}\right) + n^2$	$a = 2, \quad b = 2, \quad f(n) = n^2$ $\log_b a = \log_2 2 = 1 \rightarrow n^{\log_b a} = n$ $f(n) = n^2 = \Omega(n^{1+\varepsilon}), \quad \varepsilon = 1$ zahteva proveru regularnosti Uslov regularnosti: $a \cdot f\left(\frac{n}{b}\right) \le c \cdot f(n), \quad c < 1$ $2 \cdot \left(\frac{n}{2}\right)^2 \le c \cdot n^2$ $\frac{1}{2} \le c, \quad za\ c = \frac{1}{2} < 1$ $\rightarrow T(n) = \theta(f(n)) = \theta(n^2)$
7.	$T(n) = 25T\left(\frac{n}{3}\right) + n^2$	$a = 25, \quad b = 3, \quad f(n) = n^2$ $\log_b a = \log_3 25 \approx 2.92 \rightarrow n^{\log_b a} = n^{2.92}$ $f(n) = n^2 = o(n^{2.92-\varepsilon}), \quad \varepsilon = 0.92 > 0$ $\rightarrow T(n) = \theta(n^{\log_b a}) = \theta(n^{2.92})$
8.	$T(n) = 24.7\,T\left(\frac{n}{3}\right) + n^2$	$a = 24.7, \quad b = 3, \quad f(n) = n^2$ $\log_b a = \log_3 24.7 < 3 \rightarrow n^{\log_b a} = n^{\log_3 24.7}$ $f(n) = n^2 = o(n^{\log_3 24.7-\varepsilon}), \quad \varepsilon = \log_3 24.7 > 0$ $\rightarrow T(n) = \theta(n^{\log_b a}) = \theta\left(n^{\log_3 24.7}\right)$

Paralelno programiranje – Paralelizam

Idealni paralelni računar:
1. sekvencijalno konzistentne deljene memorije
2. homogena arhitektura
3. raspoređivanje ne košta ništa

Mere efikasnosti paralelnog algoritma:
- RAD – ukupno vreme računanja na jednom procesoru
- RASPON – najveće vreme potrebno za izvršavanje (prolazak kritičnom putanjom)

Vreme izvršavanja paralelnog procesa zavisi od:
- n – broj jedinica koje treba da se obrade
- broja raspoloživih procesora
- kako raspoređivač dodeljuje procesore linijama (putanjama)

- T_P vreme na P procesora
- T_1 vreme na jednom procesoru = RAD
- T_∞ vreme za neograničen broj procesora = RASPON (jer bi se svaka linija izvršavala na svom procesoru)

Zakon rada	$T_p \geq \frac{T_1}{p}$
Zakon raspona	$T_p \geq T_\infty$
Ubrzanje na p procesora	$\frac{T_1}{T_p}$
Linearno ubrzanje	$\frac{T_1}{T_p} = \theta(p)$
Savršeno linearno ubrzanje	$\frac{T_1}{T_p} = p$

Paralelizam:

$$\frac{T_1}{T_\infty} = \frac{RAD}{RASPON}$$

Paralelizam paralelnog algoritma je $T1/ T_\infty$, i daje informacije o:
- najveće moguće ubrzanje
- ograničenje savršenog linearnog ubrzanja (broj procesora koji je potreban)
- srednja količina rada koja se može obaviti u paraleli

Labavost paralelizma:

- Ako je labavost manja od 1, ne može se ostvari savršeno linearno ubrzanje (tj. dodatni procesori ne bi mnogo doprineli ubrazanju)
- Ako pak je labavost veća od 1, ograničavajući faktor je rad po procesoru (tj. dodatni procesori bi doprineli ubrzanju)
- Kako se labavost povećava od 1, ubrzanje sve bliže savršenom

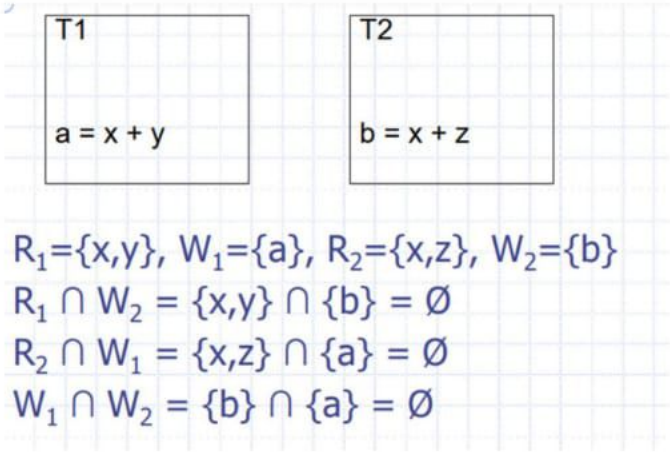
$$\frac{T_1}{T_\infty \cdot P} = \frac{RAD}{RASPON}$$

Bernštajnov uslov:

- R_i skup mem. lokacija koje čita zadatak T_i (ulaz)
- W_j skup lokacija u koje piše zadatak T_j (izlaz)

Dva zadatka, T_1 i T_2 mogu biti paralelna ako:

- Ulaz T_1 nije deo izlaza T_2
- Ulaz T_2 nije deo izlaza T_1
- Izlazi T_1 i T_2 se ne preklapaju



Teorema o gornjoj granici T_p

Koliki je max T_p?

(min T_p se može dobiti iz zakona o radu/rasponu: min $T_p = T_1/p$ min $T_p = T_\infty$)

Na P procesora, pohlepni raspoređivač izvršava paralelni algoritam sa radom T_1 i rasponom T_∞ u vremenu:

$$T_p \leq \frac{T_1}{p} + T_\infty$$

1. posledica: T_P nikada nije više od 2 puta veće od optimalnog vremena $T_p^* : T_p \leq 2T_p^*$

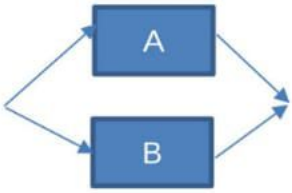
2. posledica: ako je $P << T_1/T_\infty$, tada vreme $T_p \approx T_1/P$, ili ekvivalentno, da je ubrzanje približno jednako sa P

Primeri zadataka – paralelno programiranje, paralelizam

	zadatak	Rešenje

1.	Odrediti rad i raspon primera
	
	for (int i = 0; i < n; i++)
	
	for (int i = 0; i < n; i++)
	for (int j = 0; j < n; j++)
	
	parallel_for (int i = 0; i < n; i++)
	
	parallel_for (int i = 0; i < n; i++){
	m = 0;
	for (int j = 0; j < n; j++){}
	}
		
		$\Theta(n) = RAD = RASPON$
		
		$\Theta(n^2) = RAD = RASPON$
		
		$RAD = \Theta(n)$ $RASPON = \Theta(\lg n)$
		
		$RAD = \Theta(n^2)$
		$RASPON = \max\{\Theta(\lg n), \Theta(n)\} = \Theta(n)$

2.	Originalna verzija paralelnog algoritma sa radom od 2048s i rasponom 1s je optimizovana tako da je rad sveden na 1024s, a raspon na 8s. Kako su se nakon te izmene promenile gornje i donje granice vremena izvršavanja na 32 i 512 jezgara	$T_p \leq \frac{T_1}{p} + T_\infty$ $T_{32} = \frac{2048}{32} + 1 = 65s$ $T_{512} = \frac{2048}{512} + 1 = 5s$ Optimizacija: $T_{32}` = \frac{1024}{32} + 8 = 40s$ $T_{512}` = \frac{1024}{512} + 8 = 10s$ Optimizacija je smanjila vreme izvršavanja na 32 jezgra sa 65s na 40s Optimizacija je povećala vreme izvršavanja na 512 jezgra sa 5s na 10s
3.	RAD = 33 RASPON = 8 P = 4 Da li postoji mogućnost savršenog linearnog ubrzanja?	Savršeno lin. ubrzanje: $\frac{T_1}{T_p} = p$ $\frac{33}{8 \cdot 4} = \frac{33}{32} > 1 \rightarrow$ postoji mogućnost
4.	Da li je izrazi x = a + b i y = a – b mogu da se izvršavaju u paraleli?	Use: $R_1 = \{a, b\}, \ R_2 = \{a, b\}$ Def: $W_1 = \{x\}, \ W_2 = \{y\}$ Bernštajnov uslov: važi → Moguća paralelizacija! $R_1 \cap W_2 = \emptyset$ $R_2 \cap W_1 = \emptyset$ $W_1 \cap W_2 = \emptyset$

5.	Kako se računa raspon datog grafa $T_\infty(A \cup B)$ ako su dva podgrafa povezana na sledeća 2 načina   	$T_\infty(A \cup B) = T_\infty(A) + T_\infty(B)$ $T_\infty(A \cup B) = \max\{ T_\infty(A), T_\infty(B) \}$

Primeri zadataka - optimizacija

	zadatak	Rešenje
1.	Formirati matrične prelazne oblike A = 2*54/3*X, pre i posle optimizacije koja uključuje prethodno izračunavanje vrednosti izraza	Pre optimizacije M1 * 2 54 M2 / M1 3 M3 * M2 X M4 = A M3 Posle optimizacije M1 M2 M3 * 36 X M4 = A M3
	Formirati matrične prelazne oblike A = 2*276/92*B, pre i posle optimizacije koja uključuje prethodno izračunavanje vrednosti izraza	Pre optimizacije M1 * 2 276 M2 / M1 92 M3 * M2 B M4 = A M3 Posle optimizacije M1 M2 M3 * 6 B M4 = A M3